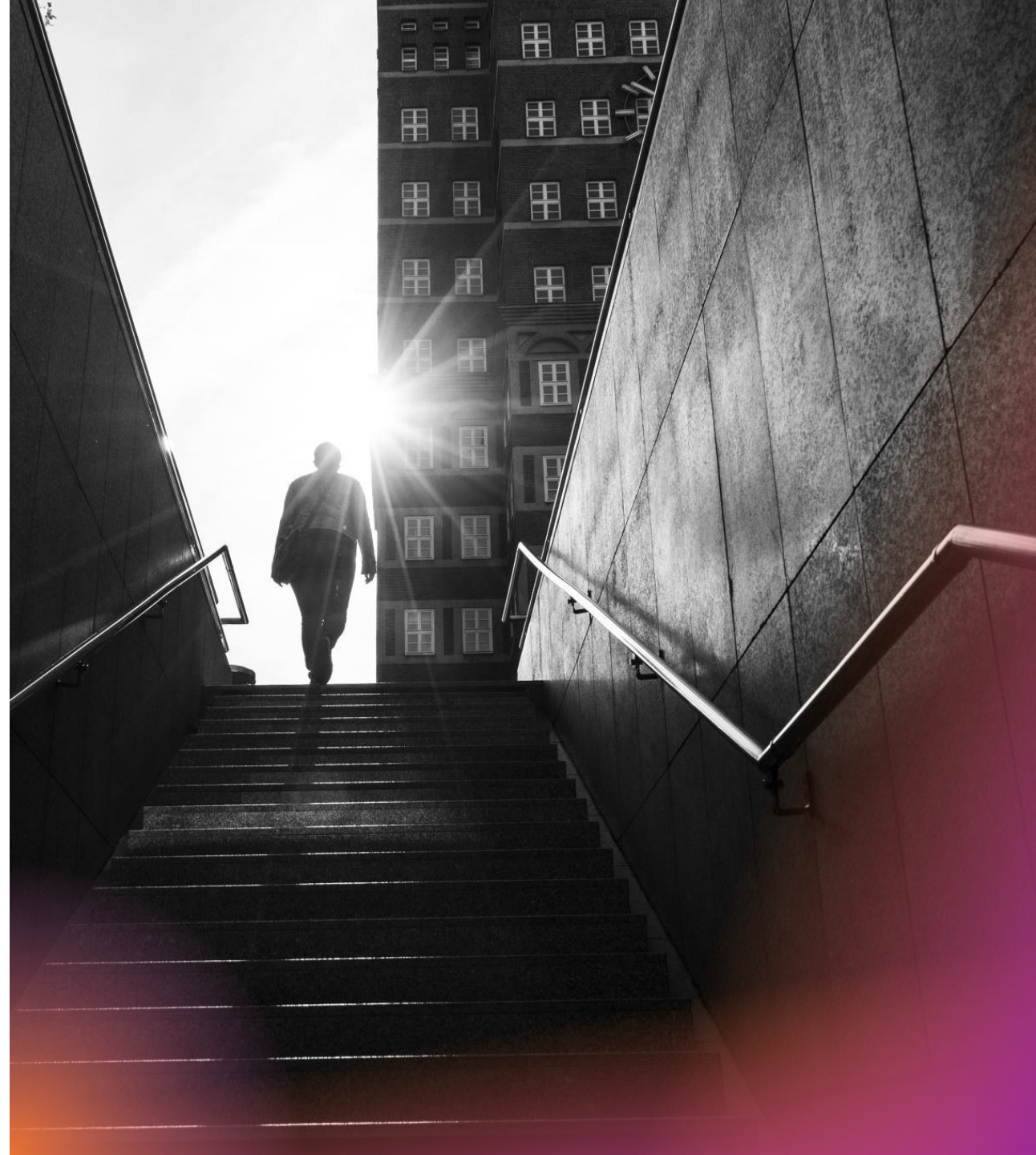


Patron Creationnel

Dr Mamadou BOUSSO



Patron Méthode de FABRIQUE

Créer une instance de plusieurs classes dérivées à partir d'une méthode

Patron Méthode de FABRIQUE



Méthode qui permet la création d'une instance de plusieurs classes dérivées



- **Problème**

Un service est offert par une hiérarchie de classes

Un objet requiert des services d'une classe dans la hiérarchie

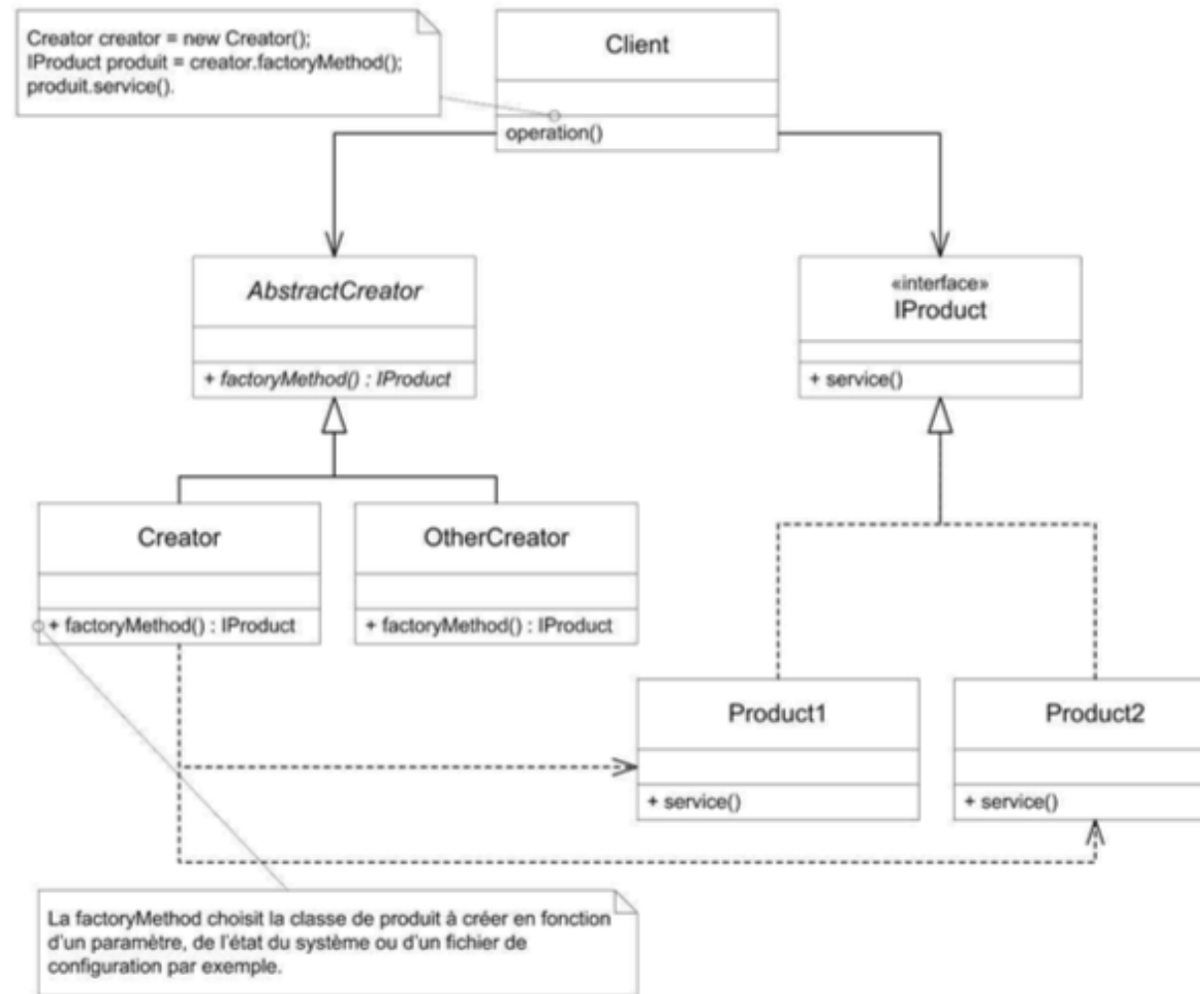
L'objet ne connaît pas la classe à instancier

- La logique de sélection de la classe concrète

Patron Méthode de FABRIQUE

- Solution
 - Encapsuler la définition du service
 - Encapsuler la fonction de sélection d'une classe particulière de la hiérarchie et son instantiation dans une méthode
 - Méthode de Fabrique
 - Le client travaille avec l'abstraction du service
 - Interface
 - Abstract Parent Class

Patron Méthode de FABRIQUE



17/01/10

Patron Méthode de FABRIQUE

Conséquences:

- Cache la logique de sélection de la classe concrète
- Uniformise l'instanciation des produits
 - Instanciation effectuée dans la méthode Fabrique
- Le client ne connaît pas la hiérarchie de classes
 - Abstraction

17/01/10

Patron Méthode de FABRIQUE

Exercice:

Module de journalisation

- Log à la console
- Log dans un fichier

17/01/10

Patron Méthode de FABRIQUE

Exercice:

Preparation de Pizza

- A la sénégalaise
- A l'américaine
- A l'italienne

17/01/10

Patron Méthode de FABRIQUE

Principe de l'inversion: Dépendre de l'abstraction. Ne jamais dépendre de classes concrètes.

De manière précise:

- Aucune variable ne doit contenir une référence à une classe concrète.
- Aucune classe ne doit hériter d'une classe concrète.
- Aucune méthode ne doit surcharger une méthode déjà implémentée dans la classe de base.



Patron FABRIQUE

Permet la création d'une instance de plusieurs familles de classes

Patron FABRIQUE

- Permet la création d'une instance de plusieurs familles de classes
- Problème
 - Plusieurs classes sont dépendantes ou sont en relations pour fournir un service
 - Le service peut être implanté de différentes façons par différentes familles de classes
 - Un objet requiert les services d'une famille de classes
 - L'objet ne connaît pas la famille de classes et les classes à instancier
 - La logique de sélection de la famille de classes

Patron FABRIQUE

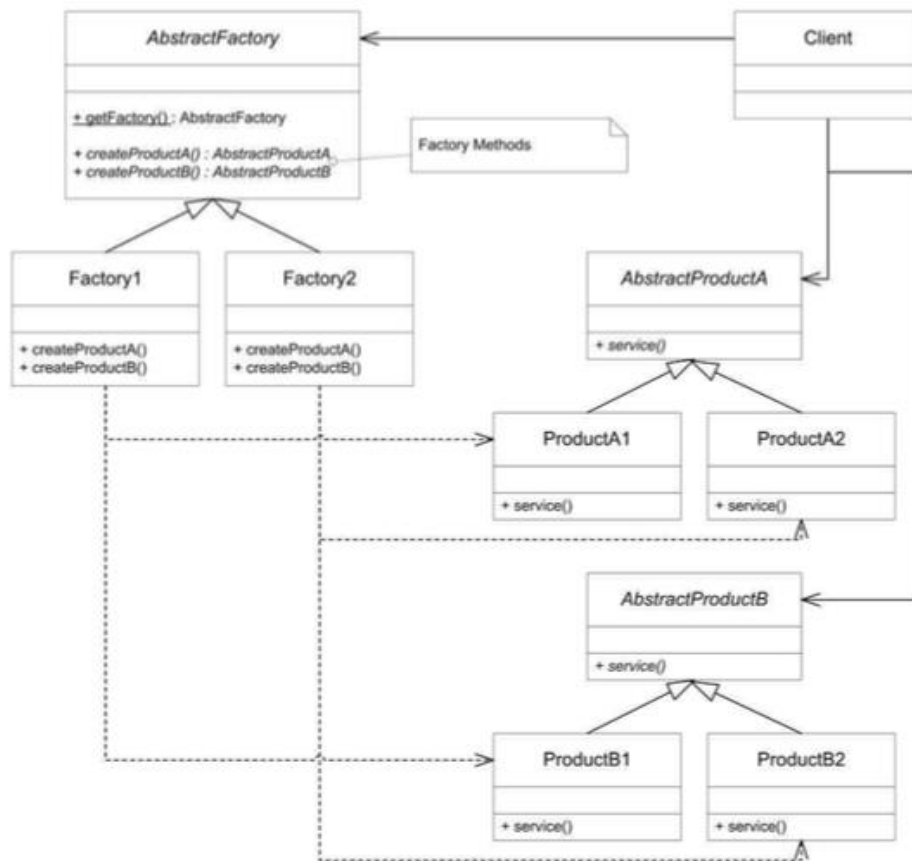
Solution

- Encapsuler les types génériques des participants au service
 - Abstractions des participants
 - Définir des méthodes de création des participants au service
 - Factory Methods
- Encapsuler la fonction de sélection d'une famille de classes particulière et l'instanciation des classes qui composent cette famille

Patron FABRIQUE

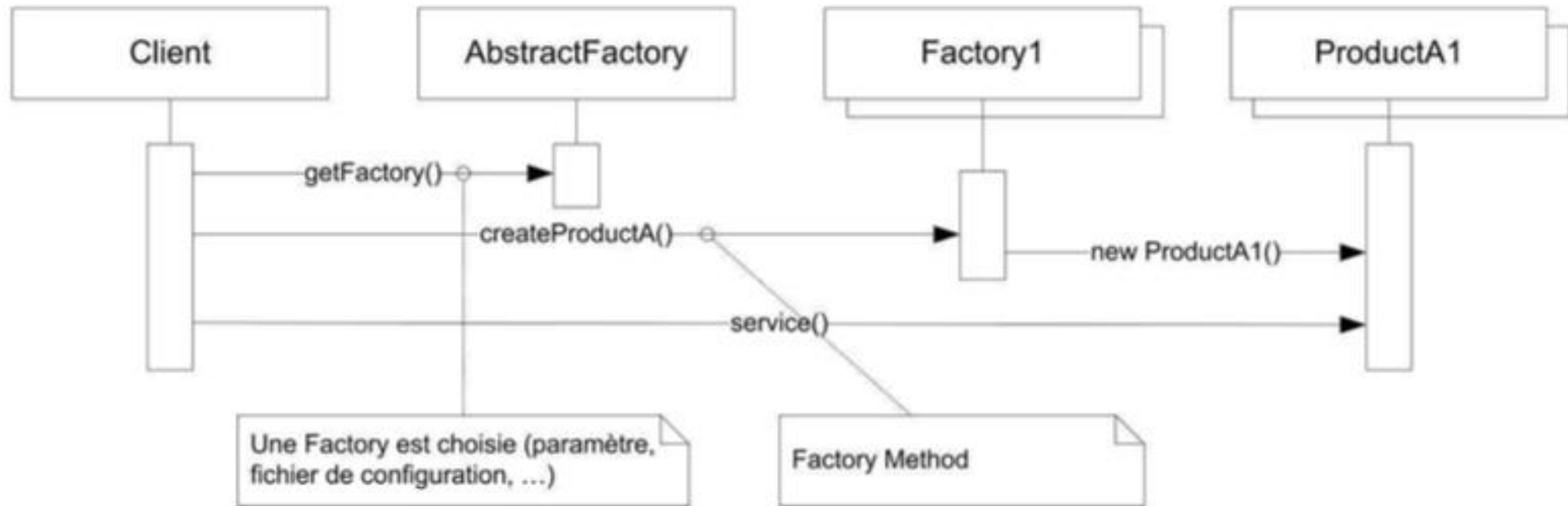
Fabrique abstraite

- Le client fait appel à une Fabrique concrète pour obtenir les objets participants au service
- Le client travaille avec une abstraction des participants
 - Interface
 - Abstract Parent Class



Patron FABRIQUE

Patron FABRIQUE



Patron FABRIQUE

Conséquences

- Cache la logique de sélection de la famille de produits
- Cache la logique de sélection du produit
- Uniformise l'instanciation des produits
 - Instanciation dans les Factory Methods
- Le client ne connaît ni les familles de classes ni la hiérarchie de classes des familles
 - Abstraction
- Permet l'échange de familles de classes
- Favorise la consistance des produits dans une famille



Patron Fabrique

Exercice:

- Système de gestion d'adresses et de numéros de téléphones
- Format Français
 - Adresse : Rue ↵ Code postal Ville ↵ Pays
 - Téléphone : 33.0X.XX.XX.XX.XX
- Format Américain
 - Adresse : Rue ↵ Ville, Région Code postal ↵ Pays
 - Téléphone : (1) XXX-XXX-XXXX

Patron Fabrique

- Exercice:
 - Supposons qu'il existe trois types de pizza:
 - Reine
 - Trois fromages
 - Oriental
 - Supposons que chaque pizza est fabriqué de manière différente au Sénégal, En France et aux USA:
 - Trois fromage: avec gruyère en France, avec fromage de chèvre au Sénégal, sans gruyère et sans fromage de chèvre aux USA

Patron Fabrique

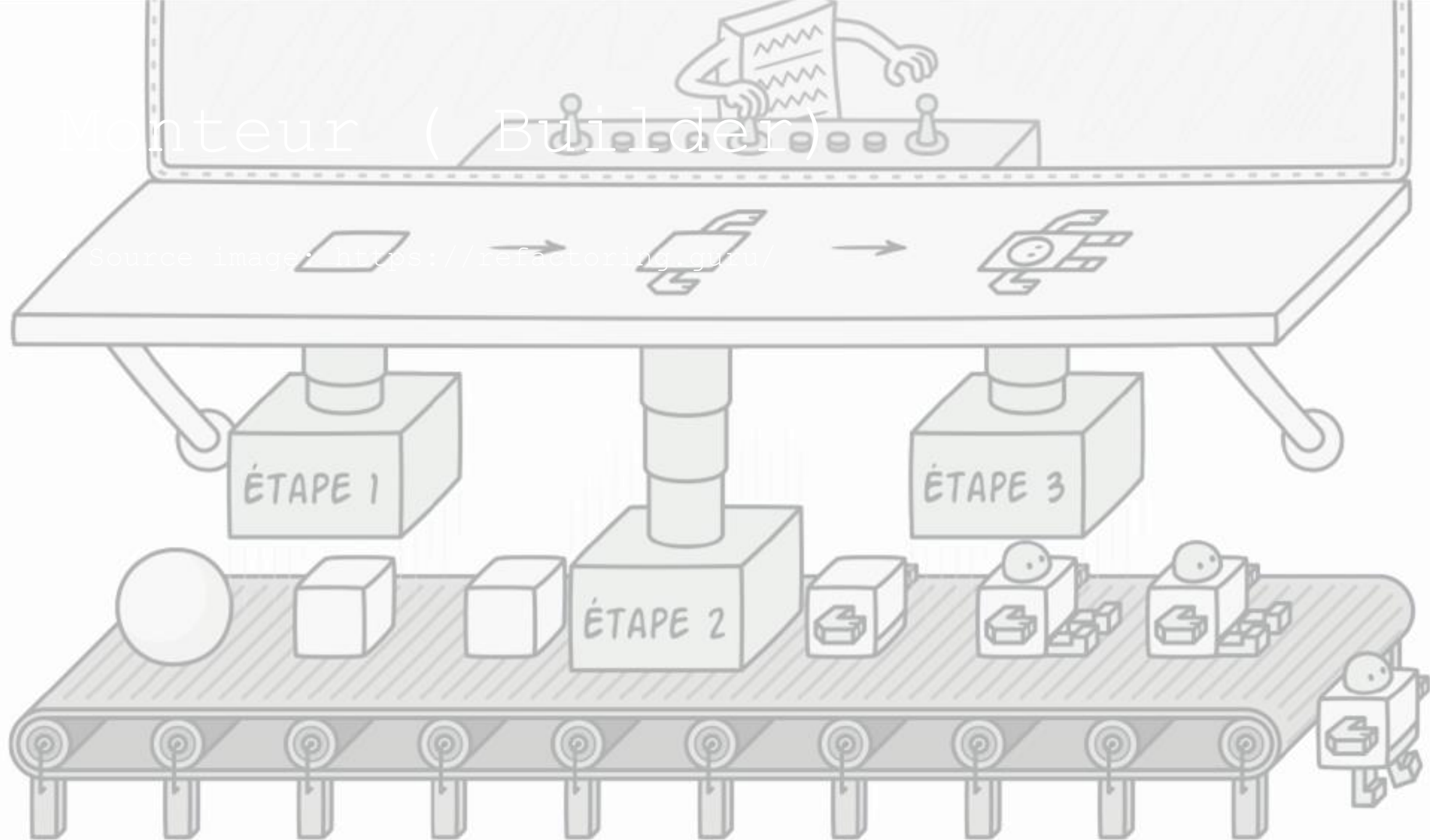
- Pizza Reine:
 - Avec jambon de porc en France, sans jambon et avec bacon aux USA et jambon de bœuf au Sénégal;
- Pizza Oriental:
 - Avec crevette au Senegal, avec gambas et sauce harira aux USA et avec calamars en France.
- Utiliser la fabrique pour créer un système de gestion des pizzas

PATRON BUILDER



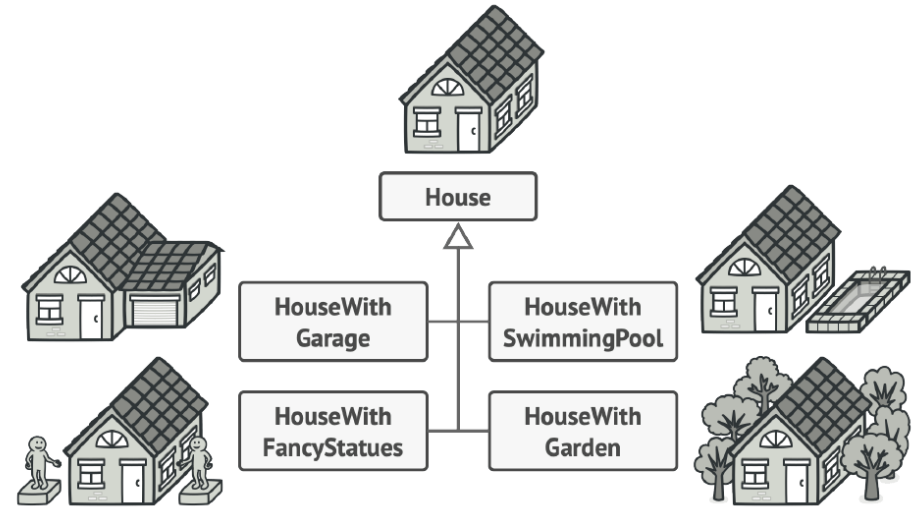
Monteur (Builder)

Source image: <https://refactoring.guru/>



Monteur (Builder)

- Problème
- Imaginez un objet complexe qui nécessite une initialisation fastidieuse, composée de plusieurs parties avec de nombreux champs et objets imbriqués.
- Le code d'initialisation va se retrouver dans un constructeur, enterré sous une pile monstrueuse de paramètres, ou encore pire : réparti un peu partout dans le code client.





Monteur (Builder)

- Réfléchissez à la façon de construire un objet House:
 - On suppose que pour fabriquer une maison de base, vous devez construire quatre murs et un sol, installer une porte, poser quelques fenêtres et bâtir un toit.
 - Mais comment procéder si vous voulez une plus grande maison avec plus de lumière, un peu de terrain et autres commodités (un système de climatisation, de la plomberie et des câbles électriques) ?



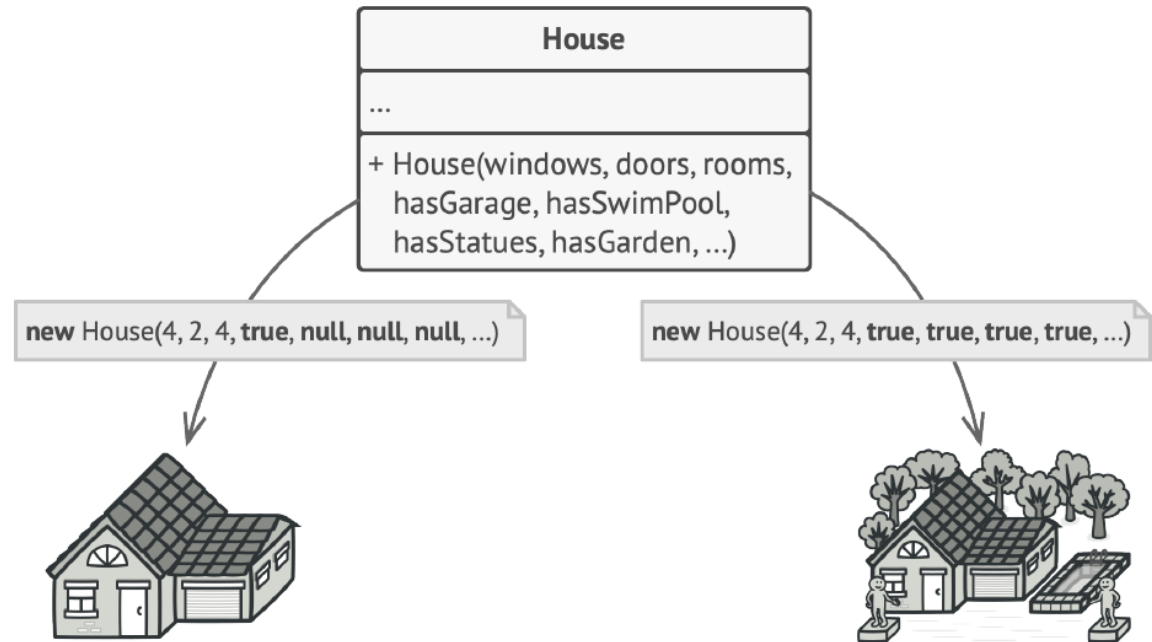
Monteur (Builder)

Mauvaise solution:

- Etendre la classe de base Maison et créer un ensemble de sous-classes pour couvrir toutes les combinaisons de paramètres.
- Mais au bout d'un certain temps, vous allez vous retrouver avec un nombre considérable de sous-classes.
 - Le moindre paramètre supplémentaire, va encore plus développer la hiérarchie.

Monteur (Builder)

- Mauvaise Solution:
Définir un constructeur monstrueux qui prend en compte tous les paramètres
- Question: Rappelez moi les principes de Liskov que cet objet ne respecte pas ?

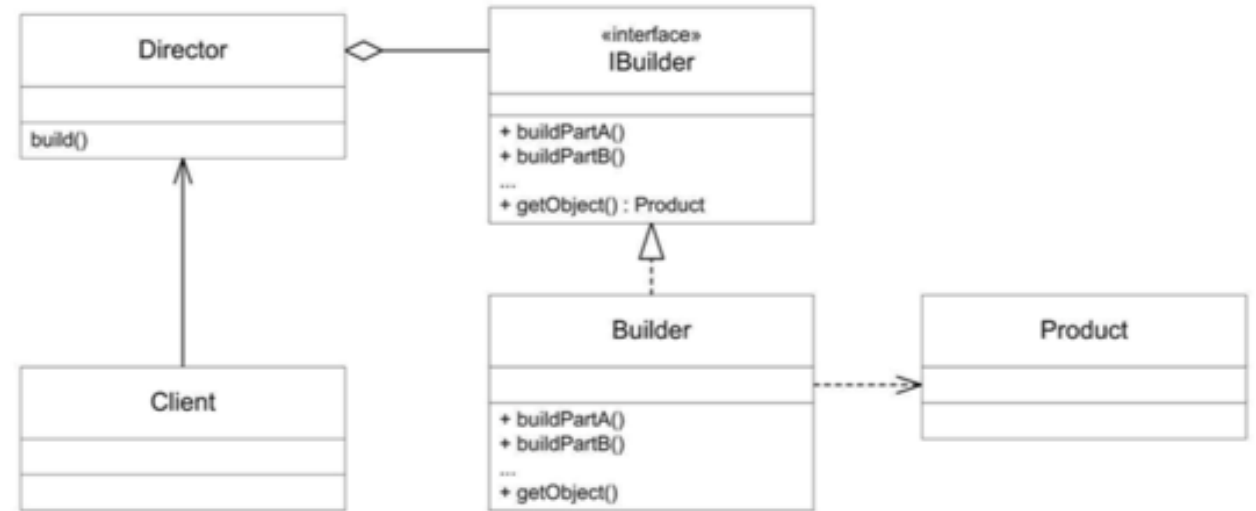


Monteur (Builder)

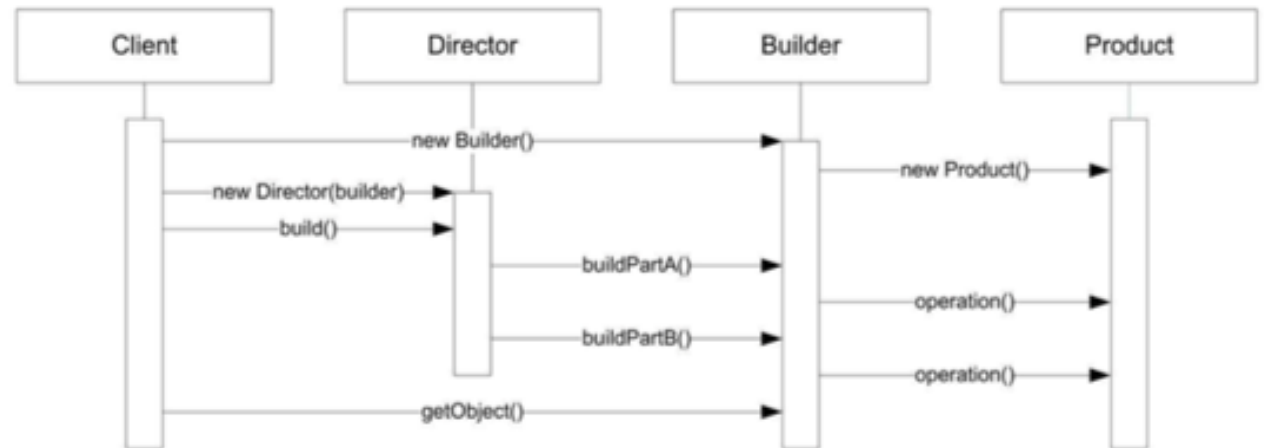
- Solution: le patron Monteur
 - Patron créationnel qui permet de construire des objets complexes à partir d'objets simples
 - La classe Monteur construit l'objet final étape par étape
 - Il est indépendant des autres objets



Modele



Modele



Monteur (Builder)



- Conséquences:
 - Améliore la modularité;
 - Rend le processus de construction indépendant des composants qui forment l'objet produit;
 - Facilite l'ajout ou la modification d'une implantation d'un processus de construction menant à une représentation particulière d'un objet;

Exercice

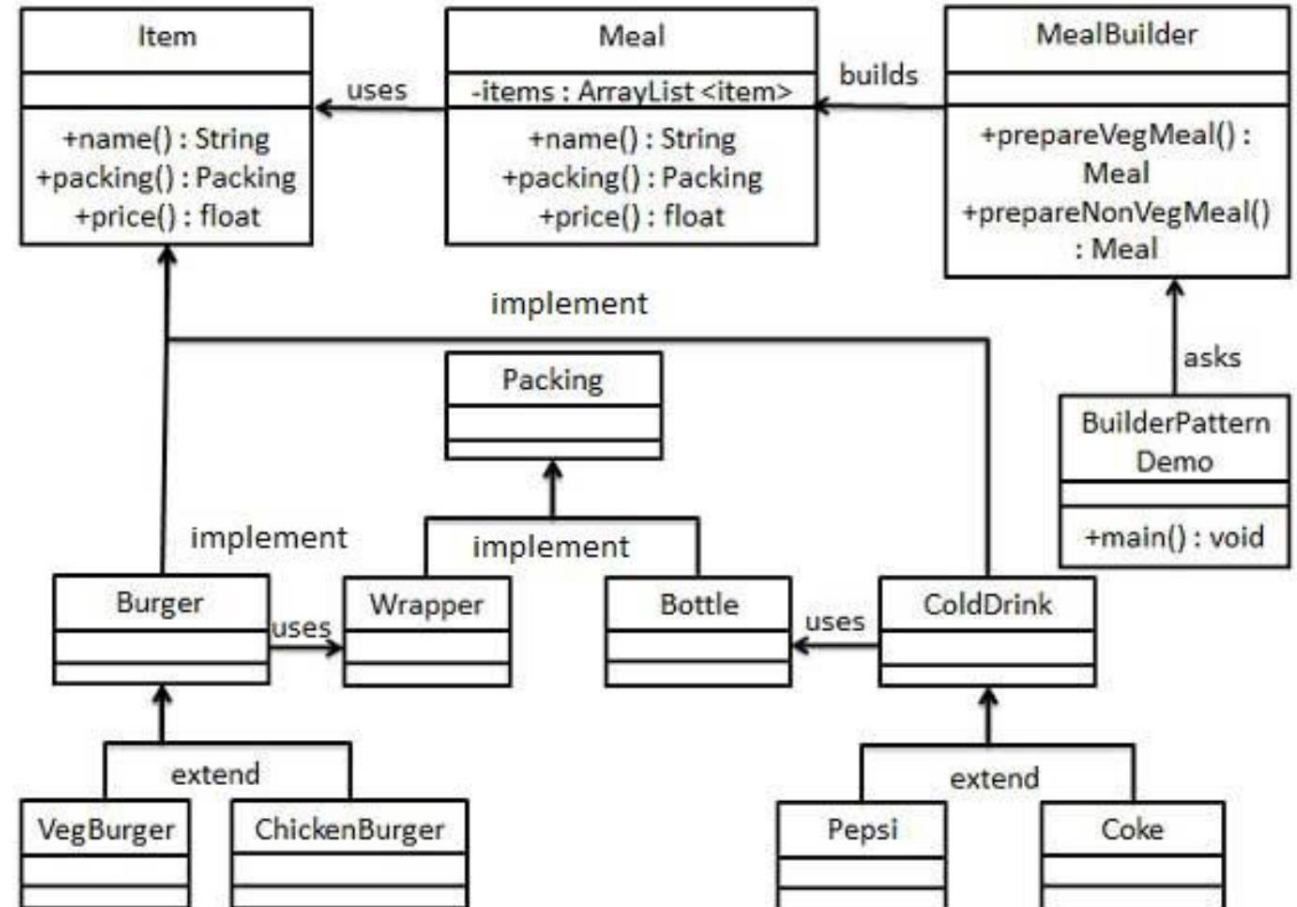
- Considérons le modèle d'un restaurant de restauration rapide où:
 - un repas typique pourrait être un hamburger et une boisson froide.
 - Le burger peut être un burger végétarien ou un burger au poulet et sera emballé dans un emballage.
 - La boisson froide peut être un coca ou un pepsi et sera emballée dans une bouteille.



Exemple

- Créer une interface Item représentant des produits alimentaires tels que des hamburgers et des boissons froides;
- Créer des classes concrètes implémentant l'interface Item
- Créer une interface Packing représentant des emballages d'aliments et des classes concrètes implémentant l'interface Packing car le hamburger serait emballé dans un emballage et une boisson froide serait emballé comme une bouteille.
- Créer ensuite une classe Meal ayant ArrayList of Item et un MealBuilder pour construire différents types d'objets Meal en combinant Item.
- BuilderPatternDemo, notre classe de démonstration.

Schéma



Exercice

On veut contruire des maisons simples composées de murs, portes, fenetres comme base.

A partir des maisons simples on veut construire des maisons simples avec piscines, jardin, sous-sol...

On suppose que les murs, portes, fenetre, piscines, jardins sont caractérisés par des dimensions(longueur, largeur et matiere)

Ecrire un builder qui permet de construire divers types de maison

A photograph of three shark fins protruding from the surface of the ocean. The largest fin is on the left, and two smaller ones are on the right. The word "FIN" is written in white capital letters across the center of the image, partially overlapping the fins. The water is blue and slightly rippled, and the sky is a pale, hazy blue.

FIN